New York Gas Price Analysis

Tayla

Introduction

This project analyzes weekly New York Harbor gasoline prices over 40 years using time series methods. The goal is to model both price dynamics and volatility using ARIMA and GARCH approaches.

Load Packages

```r
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
library(gtExtras)
```

```
Loading required package: gt
```

```r
library(gridExtra)
```

```
Attaching package: 'gridExtra'

The following object is masked from 'package:dplyr':

    combine
```

```r
library(gt)
library(grid)
library(forecast)
```

```
Registered S3 method overwritten by 'quantmod':
  method            from
  as.zoo.data.frame zoo
```

```r
library(tseries)
library(readxl)
library(xts)
```

```
Loading required package: zoo


Attaching package: 'zoo'

The following objects are masked from 'package:base':

    as.Date, as.Date.numeric


######################### Warning from 'xts' package ##########################
#                                                                             #
# The dplyr lag() function breaks how base R's lag() function is supposed to  #
# work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or       #
# source() into this session won't work correctly.                            #
#                                                                             #
# Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
# conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop           #
# dplyr from breaking base R's lag() function.                                #
#                                                                             #
# Code in packages is not affected. It's protected by R's namespace mechanism #
```

```
# Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning.  #
#                                                                             #
###############################################################################


Attaching package: 'xts'

The following objects are masked from 'package:dplyr':

    first, last
```

```r
library(FinTS)
```

```
Attaching package: 'FinTS'

The following object is masked from 'package:forecast':

    Acf

The following object is masked from 'package:gt':

    sp500
```

```r
library(zoo)
library(ggplot2)
library(scales)
library(rugarch)
```

```
Loading required package: parallel
```

Import & Clean Data

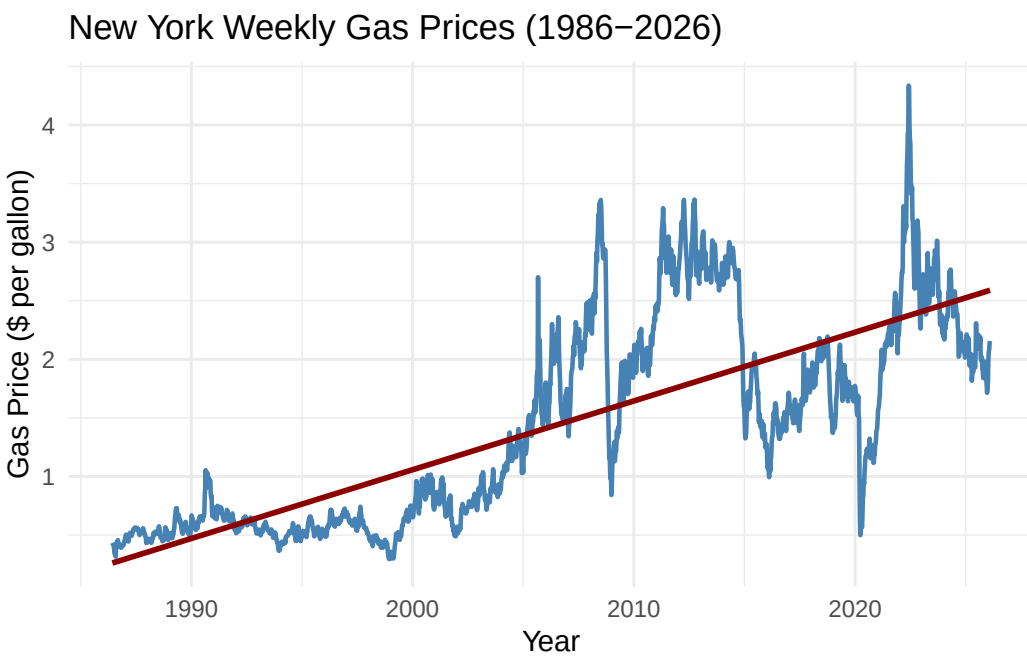
```r
# import the data
raw = read_excel("NewYork/EER_EPMRU_PF4_Y35NY_DPGw.xls", sheet = "Data 1", col_names = TRUE)

# rename columns for clarity
names(raw) <- c("date", "price")
```

```r
# the first two rows are headers in this file, so remove them
raw <- raw[-c(1,2), ]

# convert types
raw$date <- as.Date(as.numeric(raw$date), origin = "1899-12-30")
raw$price <- as.numeric(raw$price)

# convert data into xts
gas_xts =  xts(raw$price, order.by = raw$date)

# put data in data frame for plotting
gas_df <- data.frame(
  date = index(gas_xts),
  price = as.numeric(gas_xts))

# making gas prices numeric
gas_xts_num = xts(
  as.numeric(coredata(gas_xts)),
  order.by = index(gas_xts))
```

Exploratory Analysis

```r
# plotting new york gas data
ggplot(gas_df, aes(x = date, y = price)) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  geom_smooth(method = "lm", color = "darkred", se = FALSE) +
  labs(title = "New York Weekly Gas Prices (1986-2026)",
       x = "Year",
       y = "Gas Price ($ per gallon)") +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
# linear model
lm_gas = lm(gas_xts~raw$date)
```

Weekly New York gasoline prices from 1986 to 2026 show a clear long-term upward trend, rising from around 0.40c per gallon in the late 1980s to over $2.00 per gallon in recent years. However, the series also exhibits substantial fluctuations, with sharp spikes and declines reflecting major economic and energy market shocks. This combination of long-term growth and short-term volatility highlights the dynamic and unpredictable nature of gasoline prices over time.

Log Transformation

```
# check if log transformation needed using boxcox
BoxCox.lambda(as.numeric(gas_xts[,1]))
```

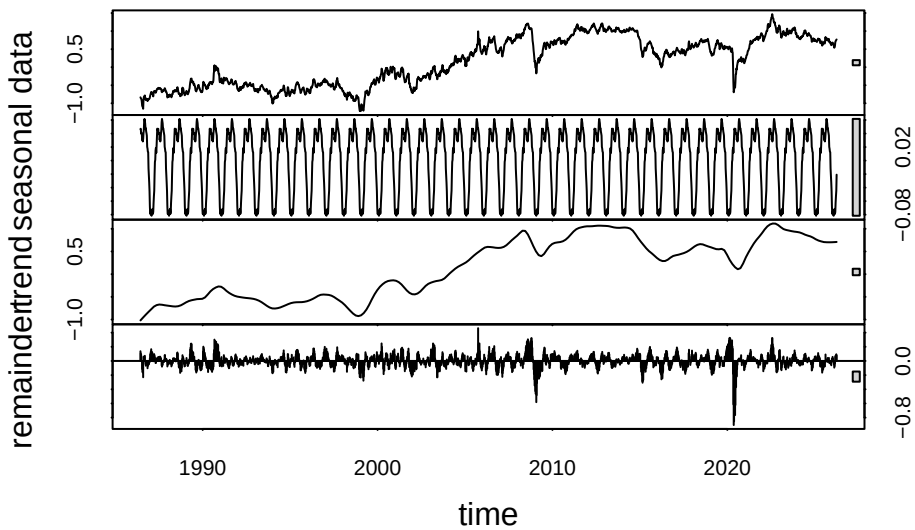
```
[1] 0.05869875
```

```
# lambda is close to 0 so log transformation recommended

# log transformation
log_gas_xts =  log(gas_xts_num)
plot(log_gas_xts, main = "Log Gas Prices")
```

A log transformation was applied to stabilize the variance of the gasoline price series and reduce the impact of large spikes. This transformation also allows changes in prices to be interpreted as percentage changes.

Seasonality

```
# Is there a repeating seasonal pattern, and what's left after removing patterns?

# convert log data to a time series
log_gas_ts <- ts(as.numeric(log_gas_xts),
                 start = c(1986, 24),
                 frequency = 52)

# STL decomposition
stl_fit = stl(log_gas_ts, s.window="periodic")
plot(stl_fit)
```




```
# STL decomposition reveals seasonal variation but is relatively small compared to the overal
```

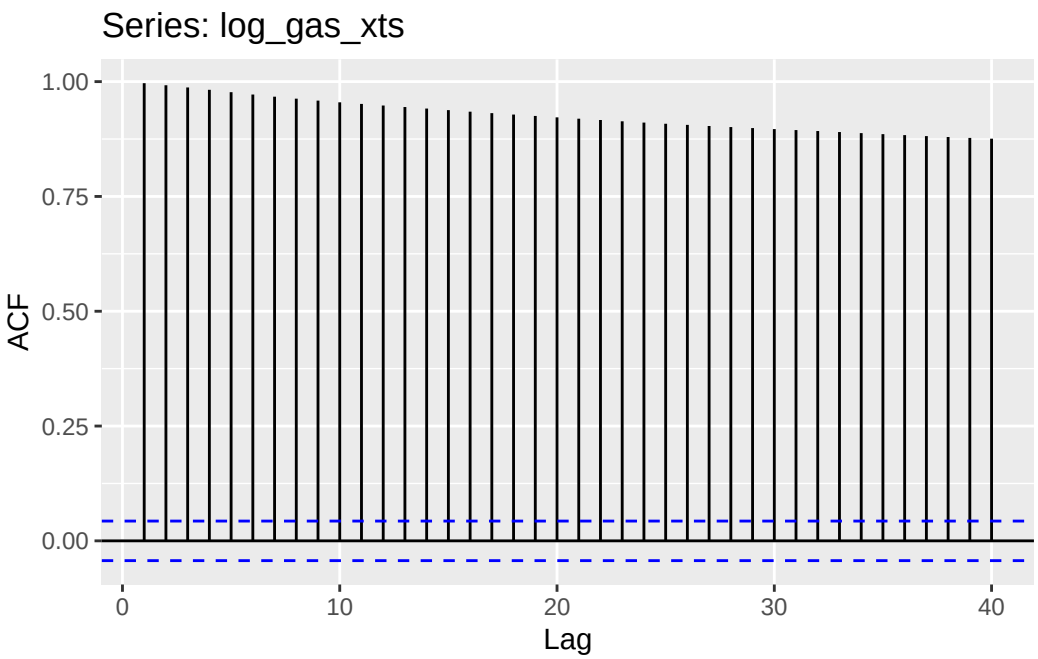
The STL decomposition separates the gasoline price series into trend, seasonal, and remainder components. The trend shows the long-term increase in gas prices over time, while the seasonal component captures small repeating patterns within each year. The remainder represents irregular movements and shocks that are not explained by the trend or seasonal effects.

Stationarity Testing

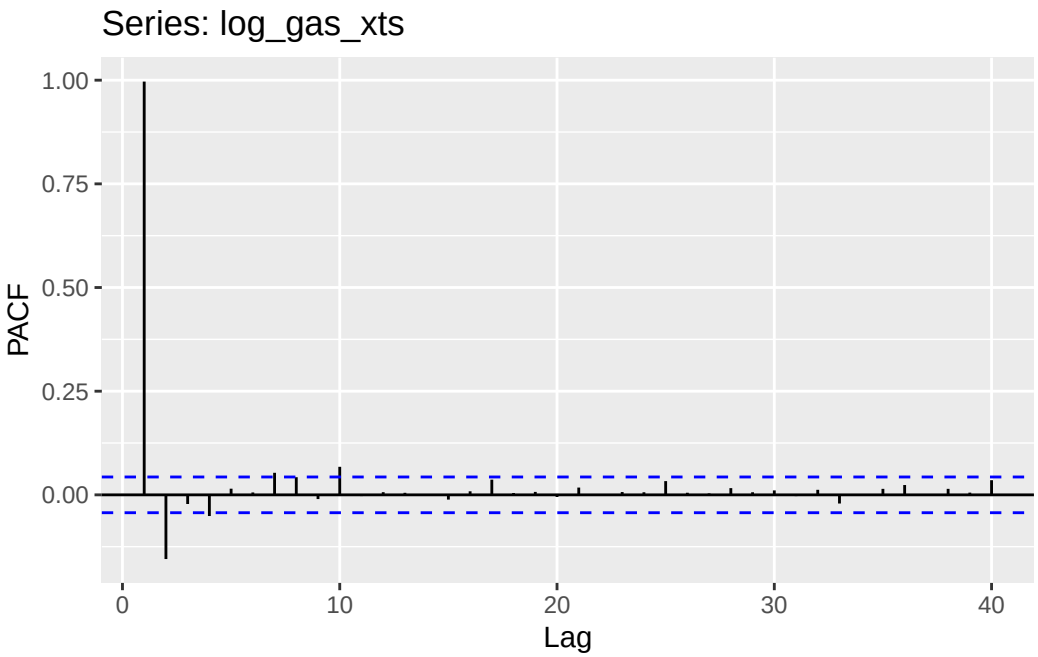
```
# is the gas data stationary?

# adf test for stationary
adf.test(log_gas_xts)
```

```
    Augmented Dickey-Fuller Test

data:  log_gas_xts
Dickey-Fuller = -3.1584, Lag order = 12, p-value = 0.09508
alternative hypothesis: stationary
```

```r
# acf for stationary
forecast::ggAcf(log_gas_xts, lag.max = 40)
```




```r
# pacf for stationary
forecast::ggPacf(log_gas_xts, lag.max = 40)
```




```
# Slow decay in the acf means there is a unit root (trend) --> not stationary
```

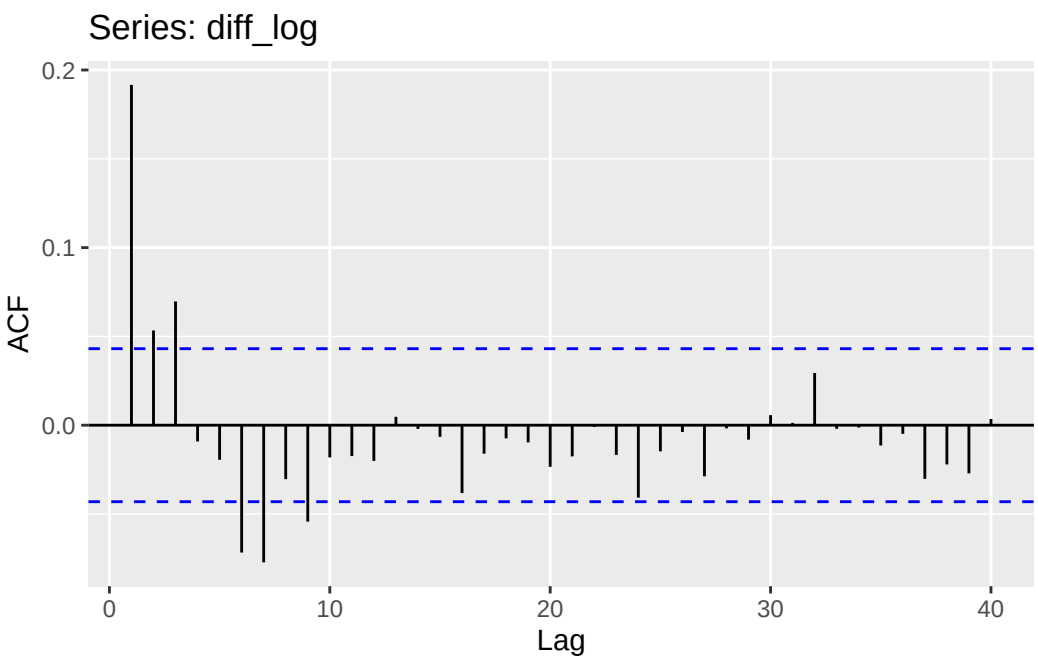
The ACF plot shows very slow decay and strong autocorrelation across many lags, indicating that the log gasoline price series is likely non-stationary. The Augmented Dickey–Fuller test fails to reject the unit root at the 5% level ($p = 0.095$), suggesting the series is non-stationary and requires differencing before modeling.

```
# take first difference of log gas data to achieve stationarity

# 1st difference
diff_log = diff(log_gas_xts)
# remove na's
diff_log = na.omit(diff_log)

# check for stationarity after 1st diff
adf.test(diff_log)
```

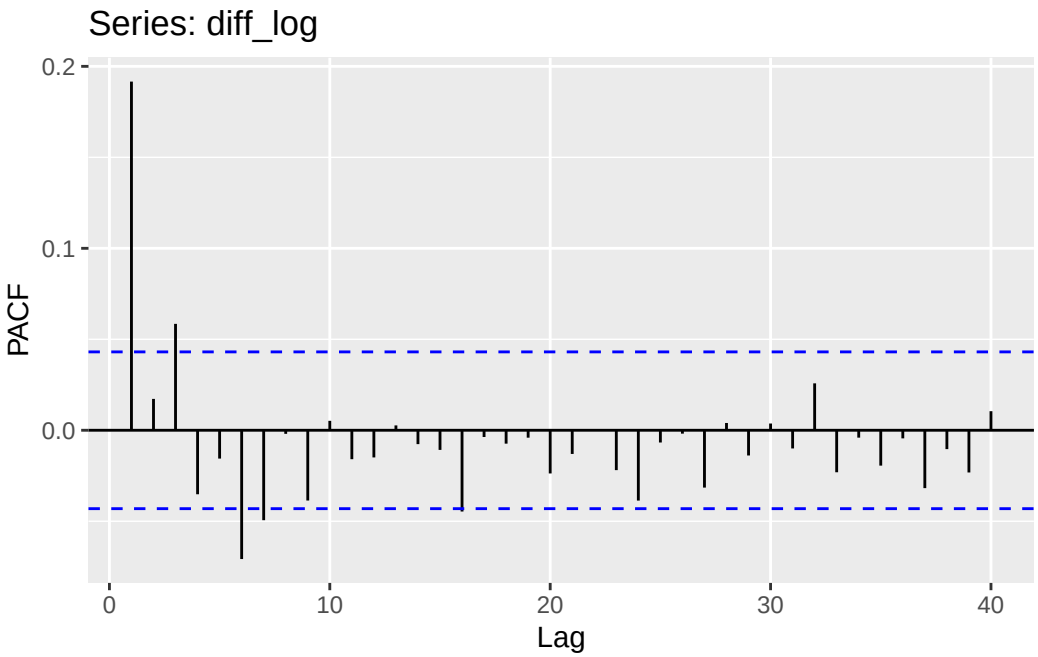
```
Warning in adf.test(diff_log): p-value smaller than printed p-value


    Augmented Dickey-Fuller Test
```

```
data:  diff_log
Dickey-Fuller = -13.637, Lag order = 12, p-value = 0.01
alternative hypothesis: stationary
```

```r
# p = 0.01 --> no unit root

# autocorrelation function
forecast::ggAcf(diff_log, lag.max = 40)
```




```r
# partial autocorrelation function
forecast::ggPacf(diff_log, lag.max = 40)
```




```
# ACF now decays quickly.The first-differenced log series is stationary. Series is I(1)
```

After first differencing the log prices, the ACF and PACF show much weaker autocorrelation with most lags within the confidence bands, indicating the series is now stationary. The Augmented Dickey–Fuller test strongly rejects the unit root hypothesis ($p = 0.01$), confirming that the differenced log series is stationary and suitable for ARIMA modeling.

```
# Model Parameter Estimates ARIMA(p,d,q) Using ACF and PACF plots:

# The patterns suggest a short-memory ARMA structure with both AR and MA components.

# The ACF shows a strong spike at lag 1 followed by rapid decay toward zero.
# a small MA component q   1, 2

# The PACF also displays a dominant spike at lag 1 with smaller short-run correlations that c
#  AR structure with several lags p   2-4

# d = 1 as we differenced once
```

ARIMA Modelling

```
# proposed arima parameters from acf and pacf
fit211 <- Arima(log_gas_xts, order=c(2,1,1))
fit311 <- Arima(log_gas_xts, order=c(3,1,1))
fit411 <- Arima(log_gas_xts, order=c(4,1,1))
fit212 <- Arima(log_gas_xts, order=c(2,1,2))

# auto arima
# If seasonal = TRUE selects D = 0 and no seasonal terms, that confirms no seasonality.

fit = auto.arima(log_gas_xts,
                  seasonal = TRUE,
                  stepwise = FALSE,
                  approximation = FALSE)

# auto arima selects arima(4,1,1) without seasonality
```

Based on patterns observed in the ACF and PACF plots, several candidate ARIMA models were estimated. An automatic model selection procedure (auto.arima) was then used to confirm the best specification, which selected an ARIMA(4,1,1) model with no seasonal components.

```
# comparing model summaries
model_comparison <- data.frame(
  Model = c("ARIMA(2,1,1)", "ARIMA(3,1,1)", "ARIMA(4,1,1)", "ARIMA(2,1,2)"),
  AIC = c(AIC(fit211), AIC(fit311), AIC(fit411), AIC(fit212)),
  BIC = c(BIC(fit211), BIC(fit311), BIC(fit411), BIC(fit212)),
  LogLikelihood = c(logLik(fit211), logLik(fit311), logLik(fit411), logLik(fit212)),
  Sigma2 = c(fit211$sigma2, fit311$sigma2, fit411$sigma2, fit212$sigma2))

# AIC (lower is better)
# BIC (lower is better)
# Log-likelihood (higher is better)
# sigma² (lower is better)

model_comparison
```

```
         Model       AIC       BIC LogLikelihood      Sigma2
1 ARIMA(2,1,1) -6712.867 -6690.326      3360.434 0.002280772
2 ARIMA(3,1,1) -6719.056 -6690.879      3364.528 0.002272852
```

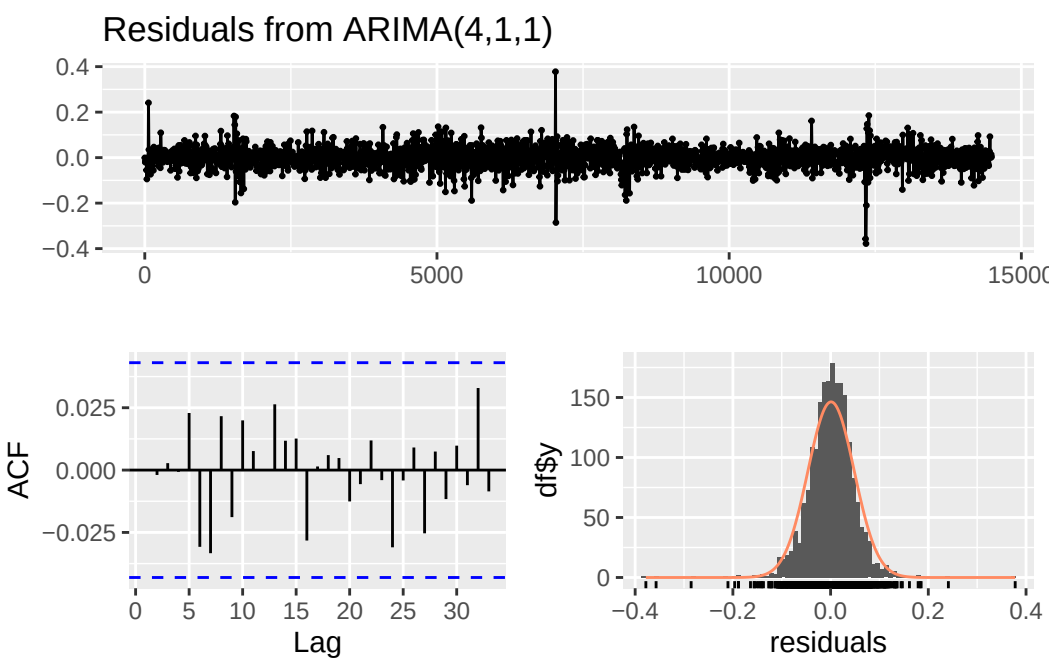
```
3 ARIMA(4,1,1) -6734.373 -6700.561      3373.186 0.002254897
4 ARIMA(2,1,2) -6716.601 -6688.424      3363.300 0.002275556
```

```
# model arima(4,1,1) preforms the best
```

The candidate ARIMA models were compared using AIC, BIC, log-likelihood, and residual variance (^{2}). Among the models tested, ARIMA(4,1,1) produced the lowest AIC and BIC values and the highest log-likelihood, indicating the best overall fit to the data . ARIMA(4,1,1) was selected to model the gasoline price series.

Residual Diagnostics

```
checkresiduals(fit)
```



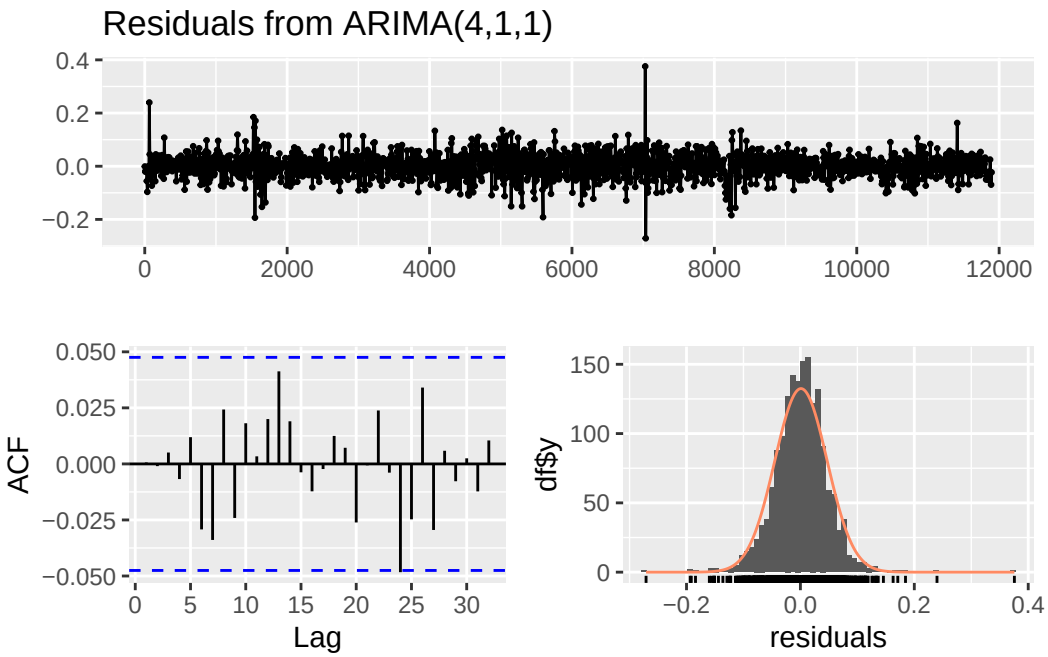

```
    Ljung-Box test

data:  Residuals from ARIMA(4,1,1)
Q* = 7.9334, df = 5, p-value = 0.1599

Model df: 5.   Total lags used: 10
```

```
# Residuals centered around zero.
# No visible pattern in time series
# Residual ACF mostly insignificant
# Ljung-Box test not significant
```

While the ARIMA model captures the mean dynamics effectively, the presence of occasional large residual spikes suggests periods of increased volatility, motivating modeling of conditional variance.

Model Validation

```
# split data into training and testing data
train =  log_gas_xts["1986/2018"]
test =  log_gas_xts["2019/2026"]

# fit model on training data
fit_train <- auto.arima(train,
                  seasonal = FALSE,
                  stepwise = FALSE,
                  approximation = FALSE)

# residuals
checkresiduals(fit_train)
```




```
    Ljung-Box test

data:  Residuals from ARIMA(4,1,1)
Q* = 6.3557, df = 5, p-value = 0.2731

Model df: 5.   Total lags used: 10
```

```
# forecast
fc = forecast(fit_train, h=length(test))

# evaluate forecasted values to
accuracy(fc, as.numeric(test))
```

```
                      ME       RMSE        MAE      MPE     MAPE      MASE
Training set 0.001003023 0.04598917 0.03435874      NaN      Inf  0.975961
Test set     0.276596827 0.42791640 0.36896392 31.27669 64.22956 10.480431
                     ACF1
Training set 0.0006488023
Test set               NA
```

```
# ME = 0.277 --> The model is systematically underpredicting or overpredicting.
# Training RMSE: 0.046 vs Test RMSE: 0.428
# MASE = 10.48 This means the model performs much worse than a naive forecast on the test dat

# mean dynamics are modeled correctly (Ljung-Box test & Residuals).
# Forecast performance is weak The model struggles to predict future shocks.
# because prices behave close to random walks.
```

The ARIMA(4,1,1) model captures the serial correlation structure of the log-differenced gasoline prices, confirmed by residual diagnostics and the Ljung–Box test. However, forecast performance deteriorates on the test set, this reflects the difficulty of predicting commodity price movements, which are influenced by unpredictable economic shocks. This motivates further modeling of conditional volatility using GARCH methods.

Time Series Cross Validation

```r
# model validation cont.
# rolling forecast
e <- tsCV(train,
          forecastfunction = function(y, h) {
            forecast(auto.arima(y), h = h) },  h = 1)

sqrt(mean(e^2, na.rm = TRUE))
```

```
[1] 0.04677417
```

```r
# on average, the one-step-ahead forecast error is about 0.047

# comparing arima model to a random walk
e_naive <- tsCV(train,
forecastfunction = function(y, h) {
  forecast(naive(y), h = h)
},
h = 1)

sqrt(mean(e_naive^2, na.rm = TRUE))
```

```
[1] 0.04689235
```

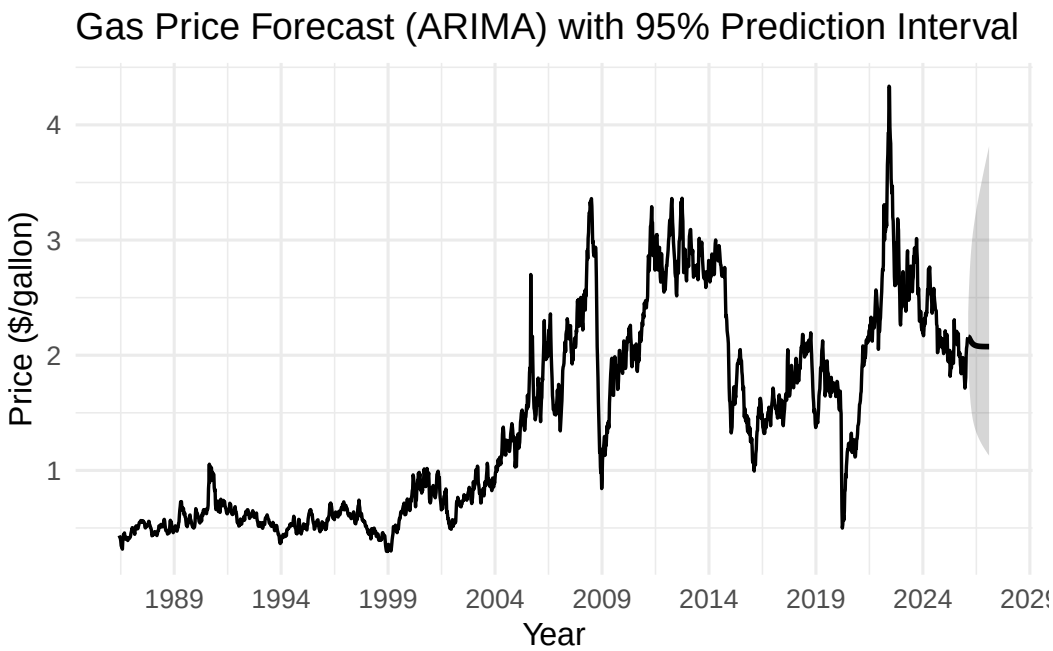
```
# forecast error of a random walk model is about 0.04689235
# ARIMA does not meaningfully outperform a random walk forecast
# difference in model errors --> (0.00012)
```

Rolling forecast cross-validation was used to evaluate one-step-ahead predictive performance. The ARIMA model produced an RMSE of 0.0468, which is nearly identical to the RMSE of a naive random walk forecast (0.0469). This indicates that while the ARIMA model captures the serial dependence structure of the data, it provides little improvement over a simple random walk for short-term forecasting. Such results are common in financial and commodity price series, which often behave close to random walks.

Forecast

```
# use model fit on all of the data,  fit411 or fit

# forecast for the next year (52 weeks)
final_fc <- forecast(fit, h = 52)

# Convert forecast to data frame
fc_df <- data.frame(
  date  = seq(from = max(gas_df$date) + 7, by = "7 days", length.out = length(final_fc$mean)
  mean  = as.numeric(final_fc$mean),
  lo95  = as.numeric(final_fc$lower[, "95%"]),
  hi95  = as.numeric(final_fc$upper[, "95%"])
)

# Back-transform because model is on log scale
fc_df$mean_p <- exp(fc_df$mean)
fc_df$lo95_p <- exp(fc_df$lo95)
fc_df$hi95_p <- exp(fc_df$hi95)

# plot
p_forecast <- ggplot() +
  geom_line(data = gas_df, aes(x = date, y = price), linewidth = 0.6) +
  geom_ribbon(data = fc_df, aes(x = date, ymin = lo95_p, ymax = hi95_p), alpha = 0.20) +
  geom_line(data = fc_df, aes(x = date, y = mean_p), linewidth = 1.0) +
  scale_x_date(date_breaks = "5 year", date_labels = "%Y") +
  labs(
    title = "Gas Price Forecast (ARIMA) with 95% Prediction Interval",
    x = "Year",
```

```
    y = "Price ($/gallon)"
  ) +
  theme_minimal(base_size = 12)

p_forecast
```



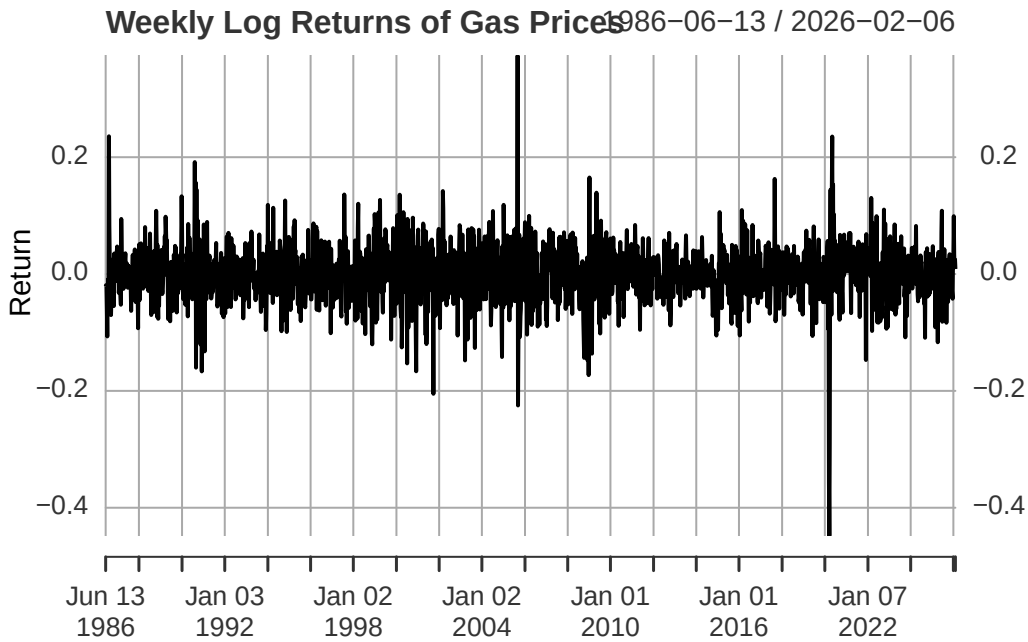

```
# short-term forecasts stay near current price
# uncertainty grows quickly
```

The ARIMA(4,1,1) model was used to generate a 52-week forecast of gasoline prices using the full dataset through 2026. The point forecast remains relatively stable around the most recent observed price level, reflecting the near random-walk behavior of gasoline prices. In such processes, future movements are largely driven by unpredictable shocks rather than deterministic trends.

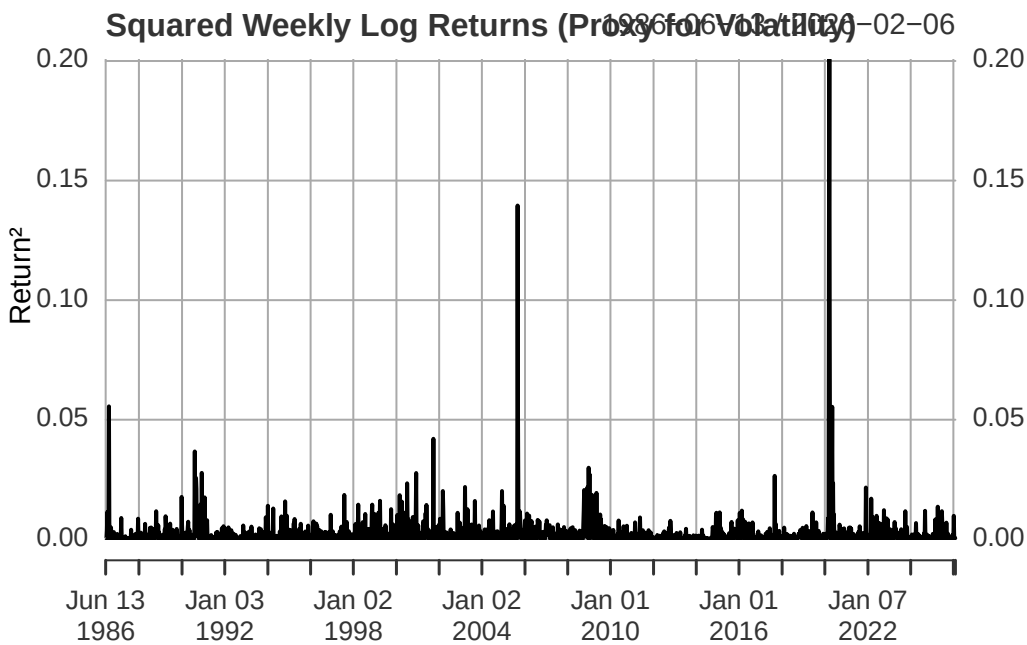
The shaded region represents the 95% prediction interval, which widens over the forecast horizon. This widening indicates increasing uncertainty in longer-term predictions, a common feature when forecasting volatile commodity prices.

Checking Volatility

```
# This plot shows the percentage change in gas prices from one week to the next.
plot(diff_log, main = "Weekly Log Returns of Gas Prices", xlab = "Year", ylab ="Return")
```




```
# there is no consistent upward or downward drift in weekly changes price movements are mostl


# Squared returns --> volatility. Large values mean large price movements.
plot(diff_log^2, main = "Squared Weekly Log Returns (Proxy for Volatility)", xlab = "Year", y
```




```
# large shocks tend to be followed by more large shocks and small by small movements
```

The weekly log return series fluctuates around zero and exhibits several large spikes corresponding to major market shocks. The absence of a persistent trend suggests that the return series is stationary.

The squared return plot reveals periods of heightened volatility, where large price movements tend to occur in clusters rather than independently. The presence of volatility clustering suggests that the variance of returns changes over time, violating the constant variance assumption of standard ARIMA models.

ARCH Test

```
ArchTest(diff_log, lags = 12)
```

```
    ARCH LM-test; Null hypothesis: no ARCH effects

data:  diff_log
Chi-squared = 476.64, df = 12, p-value < 2.2e-16
```

```r
# p-value is small ( < 0.05) --> non constant variance
# There are ARCH effects → volatility clustering exists
```

The ARCH LM test strongly rejects the null hypothesis of no ARCH effects (p < 2.2e-16), confirming the presence of volatility clustering in the differenced log series.

Garch Modelling

```r
# specify model
spec <- ugarchspec(
  mean.model = list(armaOrder = c(0,0)),  # since diff_log is already returns
  variance.model = list(model = "sGARCH", garchOrder = c(1,1)),
  distribution.model = "norm")

# fit model
garch_fit <- ugarchfit(spec = spec, data = diff_log)
show(garch_fit)
```

```
*---------------------------------*
*          GARCH Model Fit        *
*---------------------------------*

Conditional Variance Dynamics
-----------------------------------
GARCH Model : sGARCH(1,1)
Mean Model  : ARFIMA(0,0,0)
Distribution    : norm

Optimal Parameters
------------------------------------
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.000826    0.000952  0.86816 0.385306
omega   0.000195    0.000046  4.28086 0.000019
alpha1  0.129131    0.019328  6.68100 0.000000
beta1   0.788761    0.031298 25.20173 0.000000

Robust Standard Errors:
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.000826    0.001028  0.80394 0.421430
```

```
omega   0.000195    0.000080  2.43916 0.014722
alpha1  0.129131    0.042697  3.02438 0.002491
beta1   0.788761    0.061795 12.76408 0.000000

LogLikelihood : 3467.291

Information Criteria
------------------------------------

Akaike       -3.3462
Bayes        -3.3353
Shibata      -3.3462
Hannan-Quinn -3.3422

Weighted Ljung-Box Test on Standardized Residuals
------------------------------------
                        statistic   p-value
Lag[1]                      58.21 2.365e-14
Lag[2*(p+q)+(p+q)-1][2]     59.53 2.220e-16
Lag[4*(p+q)+(p+q)-1][5]     62.89 0.000e+00
d.o.f=0
H0 : No serial correlation

Weighted Ljung-Box Test on Standardized Squared Residuals
------------------------------------
                        statistic p-value
Lag[1]                      3.313 0.06874
Lag[2*(p+q)+(p+q)-1][5]     3.486 0.32542
Lag[4*(p+q)+(p+q)-1][9]     3.796 0.62350
d.o.f=2

Weighted ARCH LM Tests
------------------------------------
            Statistic Shape Scale P-Value
ARCH Lag[3]    0.0229 0.500 2.000  0.8797
ARCH Lag[5]    0.3047 1.440 1.667  0.9390
ARCH Lag[7]    0.4600 2.315 1.543  0.9819

Nyblom stability test
------------------------------------
Joint Statistic:  0.619
Individual Statistics:
mu     0.03718
```

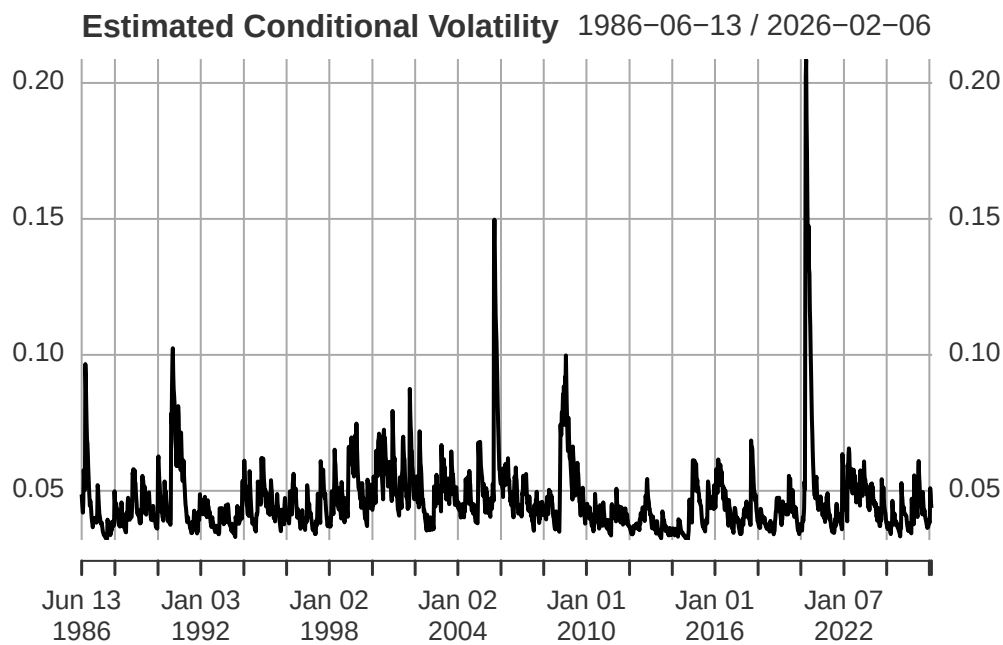
```
omega  0.17700
alpha1 0.07484
beta1  0.16447

Asymptotic Critical Values (10% 5% 1%)
Joint Statistic:         1.07 1.24 1.6
Individual Statistic:    0.35 0.47 0.75

Sign Bias Test
------------------------------------
                   t-value   prob sig
Sign Bias           0.6910 0.4896
Negative Sign Bias  0.5871 0.5572
Positive Sign Bias  0.4066 0.6843
Joint Effect        3.5964 0.3085


Adjusted Pearson Goodness-of-Fit Test:
------------------------------------
  group statistic p-value(g-1)
1    20     45.00    0.0006865
2    30     57.04    0.0014195
3    40     74.93    0.0004740
4    50     65.85    0.0543587


Elapsed time : 0.07007718
```

```
# tells us Volatility Persistence
# alpha = reaction to shocks
# beta = persistence of volatility
# alpha + beta = total volatility persistence .129 + 0.788761
# Volatility shocks decay very slowly. Gas price volatility is highly persistent.
```

The GARCH(1,1) model reveals significant volatility clustering in gasoline price returns. Both the ARCH and GARCH parameters are statistically significant, indicating that volatility responds to new shocks and remains persistent over time. The sum of the parameters (+ 0.92) suggests strong but stationary volatility persistence. Diagnostic tests indicate that the model adequately captures conditional heteroskedasticity, with no remaining ARCH effects in the residuals.

Conditional Volatility

```
sigma_vals <- sigma(garch_fit)
plot(sigma_vals, main = "Estimated Conditional Volatility")
```



The plot shows clear volatility clustering, where periods of high volatility are followed by additional periods of high volatility. After large volatility shocks, the volatility gradually declines rather than immediately returning to normal levels. Overall, the conditional volatility plot confirms that gasoline price returns periods of calm and periods of intense market turbulence.

Half Life

```
# How many weeks it takes for a volatility shock to decay by 50%.
half_life <- log(0.5) / log(0.129131+0.788761)
half_life
```

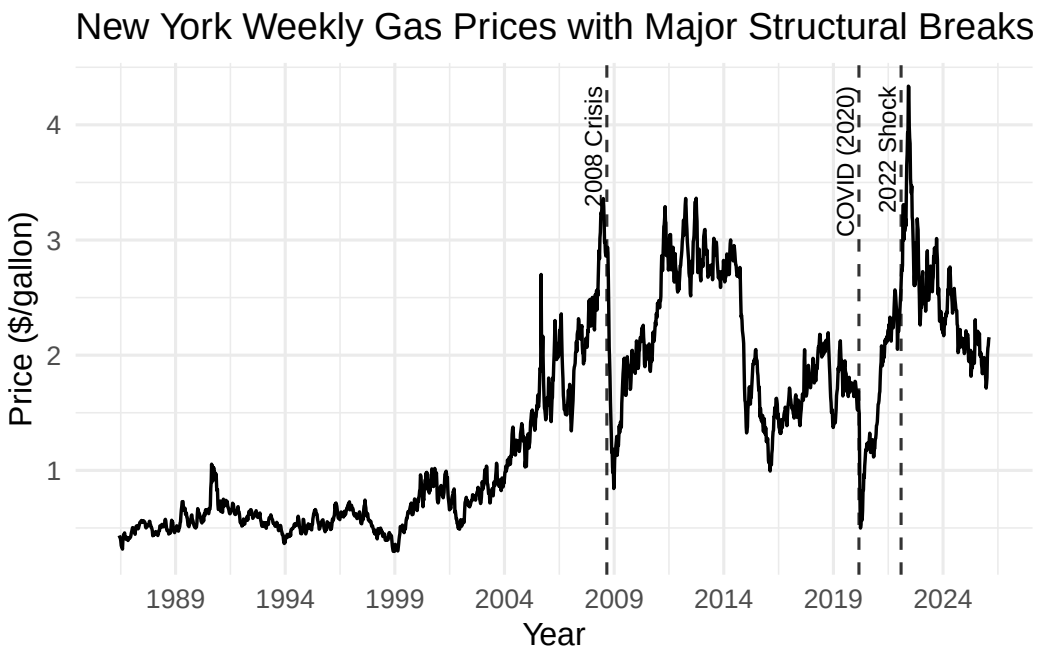
```
[1] 8.090374
```

The estimated half-life of volatility shocks is approximately 8 weeks, meaning that it takes about eight weeks for half of the impact of a volatility shock to dissipate. This indicates that

shocks to gasoline price volatility persist for several weeks before gradually returning toward normal levels.

Structural Breaks

```
# structural breaks plot
breaks <- as.Date(c("2008-09-01", "2020-03-01", "2022-02-01"))
break_labels <- c("2008 Crisis", "COVID (2020)", "2022 Shock")

p_breaks <- ggplot(gas_df, aes(x = date, y = price)) +
  geom_line(linewidth = 0.6) +
  geom_vline(xintercept = breaks, linetype = "dashed", alpha = 0.8) +
  annotate("text",
           x = breaks,
           y = max(gas_df$price, na.rm = TRUE),
           label = break_labels,
           angle = 90, vjust = -0.3, hjust = 1, size = 3) +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  labs(
    title = "New York Weekly Gas Prices with Major Structural Breaks",
    x = "Year",
    y = "Price ($/gallon)"
  ) +
  theme_minimal(base_size = 12)

p_breaks
```




```
ggsave("plot4_structural_breaks.png", p_breaks, width = 10, height = 5, dpi = 300)
```

The gasoline price series exhibits several structural breaks corresponding to major economic events. A sharp spike occurs during the 2008 financial crisis, followed by a rapid decline in prices. Another major disruption appears around 2020 during the COVID-19 pandemic, where prices collapse due to reduced demand. Prices then surge again during the 2022 energy shock, reflecting supply disruptions and rising global energy costs. These breaks highlight how external economic and geopolitical events significantly impact gasoline price dynamics.

Conclusion

This project analyzed weekly New York gasoline prices from 1986 to 2026 using time series methods. The price series was found to be non-stationary, while the log returns were stationary, allowing the mean dynamics to be modeled using an ARIMA(4,1,1) model. Forecast validation showed that gasoline prices behave similarly to a random walk, meaning future price levels tend to remain near the most recent value and are difficult to predict accurately.

Volatility analysis revealed clear volatility clustering, confirmed by the ARCH LM test. A GARCH(1,1) model was therefore used to capture time-varying variance. The results showed that volatility shocks are persistent, with a half-life of roughly eight weeks, meaning the effects of large price movements fade gradually over time. Major economic events, including the 2008

financial crisis, COVID-19 pandemic, and 2022 energy shock, correspond to significant spikes in volatility.

Overall, the results suggest that while the direction of gasoline prices is largely unpredictable, their volatility follows persistent and predictable patterns, making ARIMA–GARCH models useful for understanding energy price dynamics.